

Understanding the OWASP Top 10: A Practical Look at Five Critical Web Vulnerabilities

Introduction

Every semester I hear the phrase "web security" thrown around in lectures, but it never really clicked until I sat down and actually traced how these vulnerabilities work in practice. This report was written as part of a Cyber Security internship assignment from Vortex Tech, and the goal was simple on paper: pick five vulnerabilities from the OWASP Top 10 (2021) and explain them properly - not just define them, but understand *why* they exist, *how* someone would actually abuse them, and *what* a developer should do about it.

The OWASP Top 10 is essentially a "most wanted" list maintained by the Open Web Application Security Project, a nonprofit that studies real breach data from hundreds of organizations and ranks the vulnerability categories causing the most damage. It gets updated every few years as attack patterns shift, and the 2021 edition is still the current authoritative version referenced across the industry as of this writing.

I chose to go deep on five categories that I think every software engineer - not just security specialists - should understand cold: Broken Access Control, Cryptographic Failures, Injection, Security Misconfiguration, and Cross-Site Scripting. These aren't obscure edge cases; they show up in real breaches, real headlines, and honestly, in a lot of student projects (including some of mine) without anyone noticing until it's too late.

Table of Contents

1. [Broken Access Control](#)
 2. [Cryptographic Failures](#)
 3. [Injection](#)
 4. [Security Misconfiguration](#)
 5. [Cross-Site Scripting \(XSS\)](#)
 6. [Personal Reflection](#)
 7. [References](#)
-

Five Vulnerabilities

1. Broken Access Control

Plain Language Explanation

Think of a hotel where every guest gets a keycard. Access control is the hotel's rule that your keycard should only open *your* room - not the room next door, not the manager's office, and definitely not the security control room. Broken Access Control happens when that rule isn't actually enforced properly, so a guest's keycard ends up opening doors it was never supposed to.

In web applications, "doors" are things like user profiles, admin dashboards, invoices, or private messages. A well-built application checks, on every single request, whether the person asking for something is actually allowed to see or change it. Broken Access Control is what happens when that check is missing, weak, or can be tricked - meaning a regular user can end up doing things only an admin should be able to do, or viewing another person's private data just by asking for it the right way.

This is currently the single most common vulnerability category found in real-world applications, which honestly surprised me when I first read it, because it sounds like such a basic thing to get right.

How an Attacker Exploits It

The attack process is usually less about clever hacking tools and more about patient observation:

1. The attacker creates a normal, legitimate account on the application.
2. They log in and perform an ordinary action - say, viewing their own invoice - and watch what the web address or API request looks like (for example, a URL containing `invoice_id=1042`).
3. They notice the ID is predictable, so they simply change it to a neighboring number (`invoice_id=1043`) and resend the request.
4. If the server only checks "does this invoice ID exist?" instead of "does this invoice ID belong to *this* logged-in user?", it happily returns someone else's invoice.
5. From there, the attacker can often automate this - cycling through thousands of IDs to scrape data at scale, or discovering that certain URLs (like `/admin/dashboard`) load fine even for accounts that were never granted admin rights, simply because no one checks permissions on that page.

This category of flaw is often called an **IDOR** (Insecure Direct Object Reference) - a fancy name for a very simple idea: the system trusted an ID from the user instead of verifying ownership.

Real World Example

First American Financial Corporation (2019) is one of the clearest examples of this vulnerability at scale. The company, one of the largest title insurance providers in the United States, stored sensitive documents - bank account numbers, mortgage records, Social Security numbers, wire transaction receipts - behind URLs that used simple, sequential document ID numbers. There was no authentication check confirming that the person requesting a document was actually entitled to view it.

A security researcher discovered that by simply changing the number at the end of a document URL, anyone could view someone else's confidential financial paperwork - no login, no password, no special hacking tools required. An estimated **885 million records** dating back over a decade were exposed. The company was later fined by New York's Department of Financial Services, and the incident became a widely cited case study of how a missing authorization check, not a sophisticated exploit, can expose nearly a billion documents.

The lesson here is uncomfortable but important: this wasn't caused by an advanced attacker. It was caused by a system that never bothered to ask "does this person actually own this record?"

Prevention / Mitigation

Technique	Why It Works
Deny by default - reject every request unless explicitly permitted	Forces developers to consciously grant access rather than accidentally leaving something open
Enforce authorization checks server-side, on every request	Client-side checks (hiding a button in the UI) can be bypassed trivially by calling the API directly
Use indirect, unpredictable references (UUIDs instead of sequential IDs)	Even if authorization logic has a gap, an attacker can't simply guess the next valid ID
Centralize access-control logic instead of repeating checks in every route	Reduces the chance that one forgotten endpoint slips through without a check

Technique

Why It Works

Log and monitor access-control failures

Repeated 403/401 errors from one account are a strong signal of active probing, allowing early detection

2. Cryptographic Failures

Plain Language Explanation

Imagine writing your diary in a language only you understand, versus writing it in plain English and just hoping nobody reads it. Cryptography is that private language - it turns sensitive data into unreadable gibberish so that even if someone steals it, they can't make sense of it without the right key.

A Cryptographic Failure happens when that protection is missing, outdated, or implemented so poorly that it might as well not exist. This includes storing passwords in plain readable text, using encryption algorithms that were broken years ago, transmitting sensitive data over unencrypted connections, or using a "lock" that looks secure but has a well-known trick to pick it. It's not usually about attackers being brilliant cryptanalysts - it's almost always about a system trusting weak or improperly used encryption.

How an Attacker Exploits It

1. The attacker gains access to stored data through some other means - a database leak, a misconfigured backup, or intercepted network traffic.
2. They inspect how the sensitive fields (passwords, card numbers, personal data) are stored or transmitted.
3. If passwords were hashed using an old, fast algorithm (like unsalted MD5) instead of a slow, purpose-built one, the attacker runs the stolen hashes through freely available cracking tools and precomputed lookup tables ("rainbow tables"), reversing thousands of passwords in a short time.
4. If data was encrypted but with a weak or outdated cipher, or with the encryption key stored right next to the data itself, the attacker recovers both and decrypts everything.
5. If the connection between the browser and server wasn't properly encrypted (no HTTPS, or a poorly configured one), the attacker intercepts traffic on a shared

network and reads sensitive data as it travels - this is called a man-in-the-middle position.

The common thread is that the *idea* of protection existed, but the *execution* failed.

Real World Example

Adobe's 2013 breach remains one of the most cited cautionary tales in this category. Attackers stole a backup database containing roughly **153 million user records**, including encrypted passwords and password hints stored as plain text. The critical mistake was how the passwords were "encrypted": Adobe used a single symmetric encryption key and a mode of encryption that produced identical ciphertext for identical passwords - meaning two users with the same password had the exact same encrypted value.

This let researchers and attackers group accounts by matching ciphertext, then use the plain-text password hints sitting right next to them to guess the actual passwords for entire clusters of accounts at once. In one widely circulated example, thousands of accounts shared the encrypted value corresponding to "123456," and the hint field made it obvious. The impact went far beyond Adobe itself - because so many people reuse passwords across services, this leak fueled account takeovers on completely unrelated websites for years afterward.

The lesson was a turning point for the industry: encryption is not the same as proper password hashing, and using a weak or inappropriate cryptographic method can be almost as bad as using none.

Prevention / Mitigation

Technique	Why It Works
Hash passwords with a slow, salted algorithm (bcrypt, Argon2, scrypt) instead of encrypting or using fast hashes like MD5/SHA1	Deliberately slows down brute-force guessing, and salting means identical passwords don't produce identical stored values
Encrypt sensitive data both at rest and in transit (TLS 1.2+/1.3)	Protects data whether it's sitting in a database or moving across a network
Never store encryption keys alongside the data they protect	If a database is stolen, the key shouldn't be stolen along with it

Technique	Why It Works
Avoid deprecated algorithms (MD5, SHA1, DES, ECB mode)	These have known, publicly documented weaknesses that attackers can exploit with freely available tools
Minimize what sensitive data is even collected or retained	Data that was never stored can never be leaked - the strongest mitigation is often simply reducing exposure

3. Injection

Plain Language Explanation

Picture a librarian who takes written requests through a slot in the wall and reads them out loud exactly as written to fetch the book. Normally you'd write "Please bring me *Harry Potter*." But what if you wrote: "Please bring me *Harry Potter*. Also, unlock the back door and hand over every book in the restricted section." If the librarian can't tell the difference between a book title and an instruction, you've just tricked them into doing something they never should have done.

That's Injection in a nutshell. It happens when an application takes input from a user and passes it directly into a command, database query, or interpreter without properly separating "this is data" from "this is an instruction." The most well-known form is **SQL Injection**, where user input meant to be a search term or a login field ends up being interpreted as part of a database command.

How an Attacker Exploits It

1. The attacker finds an input field - a login form, search bar, or URL parameter - that gets used to build a database query behind the scenes.
2. They test the field with unusual characters (like a single quote ') to see if the application throws a database error, which reveals that their input is being inserted directly into a query without being treated as plain text.
3. Once confirmed, they craft input that manipulates the query's logic - for example, entering something into a login form that makes the underlying query always evaluate to "true," bypassing the password check entirely.
4. From there, attackers escalate: extracting entire database tables, dumping usernames and password hashes, or in more advanced cases, chaining injection

with server misconfigurations to execute commands directly on the underlying operating system.

5. Because this can often be automated, attackers commonly use scanning tools that test thousands of input fields across a target site in minutes, looking for the one field where sanitization was forgotten.

Real World Example

Equifax (2017) is the case study every security course eventually gets to. Attackers exploited a known vulnerability in Apache Struts, a web application framework, that allowed a specially crafted request to be interpreted and executed as a command rather than as ordinary input - a variant of the injection family of flaws. A patch for this exact vulnerability had been released by Apache months earlier, but Equifax had not applied it to the affected system.

Once inside, attackers moved through Equifax's internal network for over two months without being detected, eventually accessing systems containing sensitive records for approximately **147 million people**, including Social Security numbers, birth dates, addresses, and in some cases driver's license numbers. It became one of the most consequential data breaches in U.S. history, resulting in a settlement of up to **\$700 million**, congressional hearings, and the resignation of several senior executives, including the CEO.

What made this case so significant wasn't the sophistication of the exploit - the vulnerability and its fix were publicly known - it was the gap between "a patch exists" and "a patch gets applied." It taught the industry that injection-class vulnerabilities are rarely about needing exotic new attack techniques; they're about basic hygiene failing at scale.

Prevention / Mitigation

Technique	Why It Works
Use parameterized queries / prepared statements instead of building queries with string concatenation	The database engine treats user input strictly as data, never as executable query logic, regardless of what characters are entered
Apply strict input validation (allow-lists of expected characters/formats)	Rejects unexpected input before it ever reaches the interpreter

Technique	Why It Works
Use an ORM (Object-Relational Mapping) library correctly	Most modern ORMs parameterize queries automatically, removing the risk of manual mistakes
Apply the principle of least privilege to database accounts	Even if injection succeeds, a restricted database account limits how much damage can be done
Keep frameworks and dependencies patched promptly	Many injection-class breaches (like Equifax) exploit <i>known</i> vulnerabilities with <i>available</i> fixes - patching closes the window of opportunity

4. Security Misconfiguration

Plain Language Explanation

Imagine buying a brand-new safe for your house, but never changing it from the factory default combination (0-0-0-0), leaving the safe's door slightly ajar, and taping the combination to the front "just in case you forget." The safe itself might be excellent engineering - the problem is entirely in how it was set up.

Security Misconfiguration is exactly this, applied to servers, cloud infrastructure, and applications. It covers things like leaving default admin passwords unchanged, leaving debug modes or verbose error messages turned on in a live production system (which can leak internal details to attackers), leaving cloud storage buckets publicly accessible when they shouldn't be, or running outdated software with unnecessary features enabled. Unlike Injection or XSS, this vulnerability isn't really about a coding mistake - it's about a setup or maintenance mistake.

How an Attacker Exploits It

1. The attacker scans a target's public-facing infrastructure - servers, cloud storage, admin panels - looking for anything left in its default or careless state.
2. They check for common giveaways: default login pages still accessible, cloud storage buckets that can be listed and read without authentication, directory listings that expose file structures, or verbose error pages revealing internal software versions.

3. Once they identify outdated software or an unpatched service exposed to the internet, they cross-reference it against public vulnerability databases to find a matching, already-documented exploit.
4. If credentials were left at factory defaults or overly permissive cloud permissions were granted "temporarily" and never revoked, the attacker simply logs in or requests data directly - no exploit needed at all.
5. From an exposed but seemingly minor entry point, attackers frequently pivot deeper into internal systems that were never meant to be reachable from the outside.

Real World Example

Capital One (2019) illustrates how a single misconfiguration in cloud infrastructure can cascade into a massive breach. A former employee of Amazon Web Services exploited a misconfigured web application firewall on Capital One's cloud environment. The misconfiguration allowed a specially crafted request to trick the server into revealing temporary security credentials tied to an overly permissive cloud role - one that had far more access than it actually needed for its job.

Using those credentials, the attacker was able to access and download data belonging to approximately **106 million** credit card applicants across the U.S. and Canada, including names, addresses, credit scores, and in some cases Social Security and bank account numbers. Capital One was fined **\$80 million** by U.S. regulators and paid roughly **\$190 million** to settle a related class-action lawsuit.

The core issue wasn't a flaw in AWS itself - it was that the cloud role attached to Capital One's firewall had permissions far beyond what it needed (a violation of least privilege), and the misconfigured firewall provided the initial foothold to abuse those permissions. It's a strong reminder that cloud security is a shared responsibility, and that "the cloud provider is secure" does not automatically mean "our configuration on top of it is secure."

Prevention / Mitigation

Technique	Why It Works
Change all default credentials before deployment	Default logins are publicly documented and are among the very first things attackers try
Disable debug mode and verbose errors in production	Prevents internal details (stack traces, file paths, software versions) from being handed to attackers for free

Technique	Why It Works
Apply least-privilege permissions on cloud roles and services	Limits how much damage a compromised credential or service can actually do
Automate configuration checks (infrastructure-as-code scanning, hardening baselines)	Removes reliance on someone manually remembering every setting across every environment
Regularly audit and patch software, removing unused features/services	Reduces the "attack surface" - fewer exposed components means fewer things that can be misconfigured or exploited

5. Cross-Site Scripting (XSS)

Plain Language Explanation

Imagine a community noticeboard where anyone can pin up a note. Normally people pin harmless messages like "Lost cat, please call this number." But what if someone pinned a note that, instead of being read as text, actually *did something* the moment someone looked at it - like automatically photocopying their wallet and mailing the copy to a stranger? That's the essence of Cross-Site Scripting: an attacker manages to get their own code to run inside a webpage that other, unsuspecting users trust and visit.

XSS happens when a website takes input from one user (a comment, a username, a search term) and displays it back to other users without properly neutralizing it first. If that input happens to contain actual code instead of plain text, and the website doesn't treat it as "just text to display," the browser of every visitor who views that page will execute the attacker's code as if it were a normal, trusted part of the site.

How an Attacker Exploits It

1. The attacker finds a place on a website where user input gets displayed back to other users - a comment section, a profile bio, a search results page, a product review.
2. They submit input containing a small script instead of ordinary text, and observe whether the website displays it as executable code or safely as plain text.
3. If the script runs, they craft a more purposeful payload - commonly one that quietly sends the viewing user's session cookie to a server the attacker controls.

4. They plant this payload somewhere other users will naturally encounter it (a popular post, a public profile) and wait.
5. When victims view the page, the malicious script runs silently in their browser, using their own logged-in session. The attacker's server receives their stolen session cookie and can use it to impersonate the victim - accessing their account without ever needing their password.

The dangerous part is that the victim doesn't need to click a suspicious link or download anything; simply *viewing* a page that contains the malicious content is enough.

Real World Example

The "Samy" worm on MySpace (2005) remains one of the most famous demonstrations of XSS in the wild, precisely because of how fast and self-propagating it was. A user named Samy Kamkar discovered that MySpace's profile pages didn't properly filter out script content hidden inside profile fields. He crafted a script that, whenever anyone simply *viewed* his profile, would silently add "Samy is my hero" to the viewer's own profile and automatically send Samy a friend request - and critically, it copied itself into the viewer's profile too, so the next person who viewed *that* profile got infected as well.

Within just **20 hours**, the worm had spread to over **one million MySpace profiles**, making it one of the fastest-spreading pieces of malicious code in web history at that point. MySpace was forced to take the entire site offline temporarily to contain it. While the payload itself was harmless - a friend request and a message - it proved conclusively that XSS could be used to build genuinely self-replicating attacks entirely inside a browser, with no traditional malware or file downloads involved.

The lesson from this incident shaped how the industry thinks about user-generated content forever after: any field where users can type something that later gets displayed to others is a potential vector, and "we'll just trust what users type" is never a safe assumption.

Prevention / Mitigation

Technique	Why It Works
Escape/encode output based on context (HTML, JavaScript, URL contexts each need different encoding)	Ensures user input is always rendered as inert text, never as executable code, regardless of what characters it contains

Technique	Why It Works
Use a Content Security Policy (CSP) header	Tells the browser to only execute scripts from trusted, explicitly allowed sources, blocking injected inline scripts even if one slips through
Use modern frameworks that auto-escape by default (React, Vue, Angular)	These frameworks treat data as data by default, requiring a deliberate opt-out to render raw HTML, which reduces accidental mistakes
Mark cookies as HttpOnly and Secure	Prevents JavaScript (including injected malicious scripts) from ever reading the session cookie, even if XSS occurs
Sanitize any user-generated HTML with a trusted library rather than writing custom filtering	Custom filters are notoriously easy to bypass with clever encoding tricks; well-tested libraries are built specifically to catch these edge cases

Personal Reflection

If I'm being honest, the vulnerability that surprised me the most was Broken Access Control - not because the concept is complicated, but because of how *unglamorous* the exploit actually is. Going into this research, I expected the "scary" vulnerabilities to involve clever payloads and deep technical trickery, and in some ways Injection and XSS fit that expectation. But Broken Access Control, as shown by the First American Financial case, was just... changing a number in a URL. No tools, no scripts, no exploit code. That gap between how simple the flaw was and how catastrophic the outcome became (885 million exposed documents) genuinely reframed how I think about "serious" security risks.

What surprised me further was realizing why this keeps happening even now, despite it topping the OWASP list. It's not that developers don't know authorization matters - it's that authorization has to be checked correctly on *every single endpoint*, every single time, with no exceptions, and that kind of consistency is genuinely hard to maintain across a growing codebase, especially when new features get added under deadline pressure. It made me realize that access control isn't a feature you build once; it's a discipline you have to maintain continuously.

Researching these five vulnerabilities also taught me something I didn't expect going in: almost none of the famous breaches were caused by attackers doing something technically brilliant. Equifax exploited a publicly known bug with a published patch that

simply wasn't applied in time. Capital One's breach traced back to overly broad permissions, not a flaw in AWS. Adobe's cryptographic failure came from choosing the wrong *type* of protection, not from an attacker breaking real encryption. Even the Samy worm, as clever as it was conceptually, exploited a missing input filter, not some deep flaw in browser security. The pattern that kept repeating was fundamentals being skipped, not sophistication being lacking.

Going forward, I think this changes how I'll approach my own projects, including my final-year work. It's easy to treat "security" as a checklist item you handle at the end, but this research made it clear that access control, input handling, and configuration decisions need to be part of the design conversation from the very first day, not bolted on afterward. As someone building toward a career that sits at the intersection of security and software engineering, I think the biggest takeaway isn't any single technique from this table - it's the mindset that most breaches are failures of consistency and process, and that being a good engineer in this field means building habits that don't rely on remembering to "be careful" every single time.

References

1. OWASP Foundation. "OWASP Top 10:2021." *OWASP*, <https://owasp.org/Top10/>
2. OWASP Foundation. "A01:2021 – Broken Access Control." *OWASP*, https://owasp.org/Top10/A01_2021-Broken_Access_Control/
3. OWASP Foundation. "A02:2021 – Cryptographic Failures." *OWASP*, https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
4. OWASP Foundation. "A03:2021 – Injection." *OWASP*, https://owasp.org/Top10/A03_2021-Injection/
5. OWASP Foundation. "A05:2021 – Security Misconfiguration." *OWASP*, https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
6. OWASP Foundation. "Cross Site Scripting (XSS)." *OWASP Community Pages*, <https://owasp.org/www-community/attacks/xss/>
7. National Institute of Standards and Technology. "Guide to General Server Security" and "Digital Identity Guidelines (SP 800-63B)." *NIST*, <https://csrc.nist.gov/>
8. Cloudflare. "What Is Cross-Site Scripting (XSS)?" *Cloudflare Learning Center*, <https://www.cloudflare.com/learning/security/threats/cross-site-scripting/>

9. Cloudflare. "What Is SQL Injection?" *Cloudflare Learning Center*, <https://www.cloudflare.com/learning/security/threats/sql-injection/>
10. Microsoft. "OWASP Top 10:2021 Mitigations in Azure." *Microsoft Learn / Azure Security Documentation*, <https://learn.microsoft.com/en-us/security/>
11. U.S. Government Accountability Office. "Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach." GAO-18-559, 2018.
12. New York State Department of Financial Services. "In the Matter of First American Title Insurance Company." Consent Order, 2021.
13. U.S. Office of the Comptroller of the Currency. "Capital One Financial Corporation Consent Order," 2020.
14. Kamkar, Samy. "Technical explanation of the MySpace Worm ('Samy is my hero')." Personal archive, samy.pl, 2005.